

Assignment 3 - Report

Martin Tomassi

1 Smart Home Alarm System

1.1 Problem analysis

The Alarm system must process asynchronous messages from three independent sources: peripheral sensors, keypad inputs and timers. Because sensors and the keypad send messages without knowing the system's current state, all state logic must be centralized within the control unit. This requires state-dependent message processing, where the exact same message produces entirely different effects depending on whether the system is disarmed, armed, or in a delay state. Managing delay-based state transitions introduces the challenge of timer cancellation. During the exit delay, the system starts a countdown timer to go from disarmed to armed state: the control unit must be able to process an incoming valid pin message as a cancellation request before the timer completes the transition. When in the Armed state, the control unit must filter incoming sensor messages against the active arming mode (full or partial) to decide whether to initiate an entry delay or ignore the event. During this entry delay, the system must guarantee that receiving a valid pin message cancels the scheduled timeout, preventing an unwanted transition to the alarm state. As a final requirement, when in the alarm state, the control unit must remain responsive to keypad messages, so the user can silence the siren and disarm the system.

1.2 General strategy

The implementation uses the Actor model via Pekko Typed, organizing actors into a tree managed by `UserGuardian`. All domain logic and state transitions are centralized inside `ControlUnitActor`, while peripheral actors (`SensorActor`, `KeypadActor`, `SirenActor`) communicate exclusively with the `ControlUnitActor` and never directly with each other. All interactions, between actors, rely on asynchronous message passing, structured according to the responsibilities of each actor:

- `SensorActor` represents physical detection devices (motion or door/window sensors). When triggered, it sends a message to `ControlUnitActor` specifying which sensor and zone have detected activity.
- `KeypadActor` represents the numeric keypad. It forwards user actions (full or partial arming requests and PIN entries) directly to `ControlUnitActor`.
- `SirenActor` represents the alarm siren. It activates or deactivates the sound based on explicit messages from `ControlUnitActor`.
- `ControlUnitActor` serves as the central finite state machine operating through five states (*disarmed*, *exitDelay*, *armed*, *entryDelay*, and *alarm*), evaluating incoming messages from the peripherals and the timer in relation to the active monitoring zones.

State within `ControlUnitActor` is maintained immutably by passing contextual data as parameters between behaviours. Because an actor processes messages sequentially, sensor events, user commands and timer signals are evaluated atomically without concurrency issues. Delays rely on Pekko's `TimerScheduler` to send timeout messages, which are cancelled upon receiving a

valid PIN. System startup and shutdown are managed by **UserGuardian**, which spawns actors, schedules scenario events and terminates the actor system upon completion.

2 Odds and Evens Game

2.1 Problem analysis

The assignment requires designing a tournament-style Odds and Evens game for $N = 2^m$ players, structured into m sequential elimination rounds. From a concurrent programming perspective, the problem is defined by three main requirements. First, intra-round parallelism dictates that all $N/2$ matches within a given round are mutually independent. Because no match relies on the progress or outcome of any other match in the same round, these execution units can be executed concurrently. Second, inter-round synchronization imposes a strict barrier between consecutive rounds: a round cannot start until every match in the preceding round has concluded and produced a winner. Finally, the total absence of shared state requires that players, match referees, and the overall tournament progression coordinate exclusively through the synchronous exchange of messages. Thus, a correct concurrent architecture must provide a mechanism for distributing independent matches as concurrent tasks, a mechanism for rejoining them to enforce round boundaries, and a communication protocol based entirely on synchronous channel-based primitives.

2.2 General strategy

The implementation is structured around long-lived player goroutines, short-lived match-specific referee goroutines and a round coordinator that manages the tournament round by round. All coordination between these entities relies exclusively on synchronous message passing through unbuffered channels, eliminating the need for shared state. Players are instantiated only once at the start of the tournament and remain active for its entire duration. Each player goroutine runs a reactive loop, remaining idle until it receives a play request; at that point, it selects a random number and returns it via its dedicated channel. Winning players advance from round to round simply by passing their channel structures on to subsequent matches, keeping their goroutines active without any external management, while eliminated players wait safely in their request channel loop until the tournament ends. Each match within a round is handled by a dedicated referee goroutine spawned by the round coordinator. A referee interacts with two players using synchronous message passing: it signals both players to take their turn, waits for the numbers chosen by both, calculates the winner according to the "Odds and Evens" rule and reports the winning player to the coordinator. The round coordinator manages intra-round execution by spawning a referee goroutine for each independent pair of players. It uses a `sync.WaitGroup` along with a shared results channel to track active referees. A goroutine waits for all referee tasks to complete before closing the results channel. Because the coordinator iterates over this channel until it is drained and closed, the channel reading itself acts as a synchronisation barrier, guaranteeing that all matches in the current round conclude before the next round begins. Once the overall winner is declared, the main process initiates a graceful termination protocol by closing the request channels of all original players. This signals every long-lived player goroutine to exit its loop and terminate cleanly, preventing dangling goroutines or resource leaks.